DATABASE_ARCHITECTURE

Primary database: PostgreSQL 16-alpine **Adapter:** `@payloadcms/db-postgres` **Schema management:** Auto-migrate via Payload (dev: `push: true`; prod: explicit migrations) **Connection:** `postgres://securityblogs:securityblogs@db:5432/securityblogs` (default in docker-compose)

---

1. Entity-relationship diagram (logical)

2. Table inventory (auto-generated by Payload migrations)

Table	Origin	Primary key	Key columns	Indexes
`users`	Users collection	id (uuid)	email (unique), role, isActive, lastLoginAt, loginCount, name, hash, salt, login_attempts, lock_until	email unique
`media`	Media collection	id	filename, mime_type, filesize, sizes (JSONB), alt_text, caption, credit, uploaded_by_id FK	filename idx

Table	Origin	Primary key	Key columns	Indexes
pages	Pages collection	id	title, slug (unique), status, published_at, modules (JSONB), ai_visibility (JSONB)	slug unique
pages_v	Versions of Pages	id	parent_id FK → pages, snapshot JSONB, created_at	parent_id idx
services	Services collection	id	title, slug (unique), short_desc, intro, hero_badge, accent_color, status, sort_order, capabilities (JSONB), process_steps (JSONB), stats (JSONB), faqs (JSONB), benefits (JSONB), ai_visibility (JSONB)	slug unique
services_v	Versions of Services	id	parent_id FK → services	
case_studies	CaseStudies collection	id	headline, slug (unique), client_name, partner_id FK, service_id FK, summary, body (JSONB Lexical), results (JSONB), status, published_at	slug unique
case_studies_v	Versions of CaseStudies	id	parent_id FK	
partners	Partners collection	id	name, slug (unique), type, region, summary, body (JSONB Lexical), case_study_id FK	slug unique
partners_services	M2M join (Partners.services)	composite	partner_id, service_id	both FKs
posts	Posts collection	id	title, slug (unique), category, excerpt, body (JSONB Lexical), author_id FK, status, published_at, reading_minutes, view_count, ai_visibility (JSONB)	slug unique, category idx

Table	Origin	Primary key	Key columns	Indexes
posts_v	Versions of Posts	id	parent_id FK	
leads	Leads collection	id	display_title (computed), name, email, phone, company, subject, message, source, status, lifecycle_stage, priority, assigned_to_id FK → users, value_estimate, won_at, won_value, lost_at, lost_reason, lost_notes, ip, user_agent, referrer, honeypot, turnstile_token, page_url, extras (JSONB), timeline (JSONB), tags (JSONB), created_at	status idx, assigned_to_id idx
redirects	Redirects collection	id	from_path (unique), to_path, status_code, is_regex, is_active, hit_count, last_hit_at, note	from_path unique, is_active idx
settings	Settings global	id (always 1)	brand_*, contact_*, social (JSONB), footer_* (JSONB), seo_*, analytics_*, booking_slots (JSONB), cookie_*, maintenance_*	id PK (single row)
payload_preferences	Payload internal	id	user_id, key, value	per-user prefs
payload_migrations	Payload internal	id	name, batch	migration log
payload_locked_documents	Payload internal	id	global_slug or relation_to + relation_id	edit-lock tracking

3. Foreign keys

From	To	On delete	Notes
media.uploaded_by_id	users.id	SET NULL	Allow user deletion without losing media

From	To	On delete	Notes
`posts.author_id`	`users.id`	RESTRICT	Block user deletion if they have posts
`case_studies.partner_id`	`partners.id`	SET NULL	Allow partner deletion
`case_studies.service_id`	`services.id`	SET NULL	Allow service archive
`partners.case_study_id`	`case_studies.id`	SET NULL	Optional featured study
`partners_services.partner_id`	`partners.id`	CASCADE	Drop join rows when partner removed
`partners_services.service_id`	`services.id`	CASCADE	Drop join rows when service removed
`leads.assigned_to_id`	`users.id`	SET NULL	Allow assignee deletion
`pages_v.parent_id`	`pages.id`	CASCADE	Versions live with parent
(same pattern)	for `services_v`, `case_studies_v`, `posts_v`	CASCADE	

4. JSONB columns (storage of Payload's array + group + blocks fields)

Column	Shape	Reason
`pages.modules`	`[{ blockType, ...blockFields }, …]`	Polymorphic blocks
`pages.ai_visibility`	`{ entityName, primaryTopic, targetEngines[], keywordsToWin[], … }`	Reusable group
`services.capabilities`	`[{ title, description, previewVariant }, …]`	Array of nested objects
`services.process_steps`, `services.stats`, `services.faqs`, `services.benefits`, `services.ai_visibility`	similar	
`case_studies.body`	Lexical JSON state	Rich text
`case_studies.results`	`[{ metric, value, note }, …]`	
`posts.body`	Lexical JSON state	
`posts.ai_visibility`	group	
`leads.extras`	`{ ... }`	Source-form-specific
`leads.timeline`	`[{ at, byUser, type, note }, …]`	CRM events

Column	Shape	Reason
`leads.tags`	`[{ tag }, …]`	
`media.sizes`	`{ thumbnail: {url,w,h}, small: …, … }`	Image variants
`settings.*`	tabbed groups	

5. Indexes (declared in schemas)

Index	Type	Columns	Reason
`users_email_uq`	UNIQUE	(email)	Login lookup
`media_filename_idx`	BTREE	(filename)	Find by name
`pages_slug_uq`	UNIQUE	(slug)	Route resolution
`services_slug_uq`	UNIQUE	(slug)	
`case_studies_slug_uq`	UNIQUE	(slug)	
`partners_slug_uq`	UNIQUE	(slug)	
`posts_slug_uq`	UNIQUE	(slug)	
`posts_category_idx`	BTREE	(category)	Category listing
`posts_published_at_idx`	BTREE	(published_at DESC)	Sort recent
`leads_status_idx`	BTREE	(status)	Pipeline filter
`leads_assigned_to_idx`	BTREE	(assigned_to_id)	"My leads" view
`leads_source_idx`	BTREE	(source)	Source analysis
`redirects_from_path_uq`	UNIQUE	(from_path)	Middleware lookup
`redirects_active_idx`	BTREE	(is_active)	Middleware filter

6. Connection pool

```
postgresAdapter({
  pool: {
    connectionString: process.env.DATABASE_URI,
    // recommended for production:
    max: 20,                        // tune per VPS RAM
    idleTimeoutMillis: 30000,
    connectionTimeoutMillis: 2000,
  },
  push: process.env.NODE_ENV !== 'production',   // dev: auto-migrate; prod: explicit
})
```

7. Backup strategy (Phase D)

Cadence	What	Where	Retention
Nightly	`pg_dump --no-owner --no-acl --format=custom`	Encrypted, pushed to S3 / Backblaze	30 days hot, 90 days cold
Hourly	WAL archive (if PITR needed)	Same	7 days
Daily	Media-uploads tar	Same	30 days
Weekly	Full system snapshot (VPS provider's snapshot)	Provider	4 snapshots

Restore drill (run monthly)

```
# Spin a fresh staging VM
ssh staging
pg_restore --no-owner --dbname=securityblogs --clean s3://backups/2026-06-13.dump
psql -c "SELECT count(*) FROM users; SELECT count(*) FROM pages;"
# expect non-zero counts matching production
```

8. Migration workflow

Dev

```
cd cms
pnpm payload migrate         # creates / updates tables
pnpm seed:all                # populates Services, Pages, etc.
```

Production

```
# 1. Take backup BEFORE migrating
pg_dump --format=custom --file=pre-migrate-$(date +%F).dump $DATABASE_URI

# 2. Migrate
NODE_ENV=production pnpm payload migrate

# 3. Verify
psql $DATABASE_URI -c "\dt"   # confirm new tables exist

# 4. Rollback if needed
pg_restore --clean --dbname=securityblogs pre-migrate-2026-06-13.dump
```

9. Future tables (Phase E — SaaS platform)

Table	Purpose
`tenants`	Multi-tenant boundary (per-customer workspace)
`subscriptions`	Stripe subscription state
`usage_records`	Metered billing (API calls, AI score runs)
`api_keys`	Customer-facing API keys with scopes
`webhooks`	Customer webhook endpoints + delivery status
`audit_log`	Cross-system audit trail (who did what, when)
`ai_scores`	Brand visibility scores (Ahrefs + DataForSEO + cache)
`ad_metrics_snapshot`	Cached Supermetrics pulls for live widgets

Table	Purpose
`feature_flags`	Per-tenant feature gating

10. Performance considerations

Concern	Mitigation
Lexical body JSON column scans	Avoid full-text search on body; use a separate `search_index` table or external (Meilisearch)
Leads timeline append-only growth	Move >2yr-old rows to `leads_archive` partition
Pages with 50+ blocks → wide JSONB	Acceptable; Postgres handles MB-sized JSONB efficiently
Concurrent draft autosave (2s interval × N editors)	Connection pool max=20 should handle ~10 concurrent editors
`redirects` queried on every request	Middleware caches 60s; fall-back to in-memory stale on CMS outage

---

End of DATABASE_ARCHITECTURE.md